

The core of the LLM is a probabilistic function which maps the representation of a set of tokens to a distribution over the possible tokens that can appear next.

$$LM : [t_0, \dots, t_{n-1}] \rightarrow p(t_n | t_{i < n})$$

This prediction is called a language model. It is formally equivalent to learning a distribution over all possible token sequences: $p([t_0, \dots, t_s])$ because of the chain rule:
 $p(t_0, t_1, \dots, t_n) = p(t_0) \times p(t_1, \dots, t_n | t_0)$

This is the reason a Language Model is called a "generative model" (which is the name of probabilistic models which learn a distribution over a complex set of events).

Language models (LMs) optimize the following likelihood objective on a large text corpus U (considered as a long sequence of tokens):

$$L_1(U) = \sum_i \log(p(u_i | u_{i-k}, \dots, u_{i-1}; \theta))$$

where k is the size of a context window. This objective corresponds to the cross-entropy loss function over all windows of k tokens observed in the corpus. Maximizing this objective means that the model can predict the token u_i out of the tokens in the vocabulary of the tokenizer in an accurate manner (the model predicts a high probability for the word that is actually observed in the corpus). LMs are evaluated on a test corpus with the metric of *perplexity* which measures the amount of uncertainty in predicting the observed words of the corpus.

Original LMs have been trained on **n-gram methods** (1970s-2000s). The idea of using neural networks to learn a LM has been introduced in [Bengio et al, 2000].

The abstract of this seminal paper is important to read, as it introduces a key concept behind the success of LLMs: **representation learning**:

A Neural Probabilistic Language Model

Yoshua Bengio, Réjean Ducharme, Pascal Vincent, Christian Janvin, JMLR 2000

<https://www.semanticscholar.org/paper/A-Neural-Probabilistic-Language-Model-Bengio-Ducharme/6c2b28f9354f667cd5bd07afc0471d8334430da7>

A goal of statistical language modeling is to learn the joint probability function of sequences of words in a language. This is intrinsically difficult because of the curse of dimensionality: a word sequence on which the model will be tested is likely to be different from all the word sequences seen during training. Traditional but very successful approaches based on n-grams obtain generalization by concatenating very short overlapping sequences seen in the training set. We propose to fight the curse of dimensionality by **learning a distributed representation for words which allows each training sentence to inform the model about an exponential number of semantically neighboring sentences**. The model learns simultaneously (1) **a distributed representation for each word** along with (2) **the probability function for word**

A Neural Probabilistic Language Model

sequences, expressed in terms of these representations. Generalization is obtained because a sequence of words that has never been seen before gets high probability if it is made of words that are similar (in the sense of having a nearby representation) to words forming an already seen sentence. Training such large models (with millions of parameters) within a reasonable time is itself a significant challenge. We report on experiments using neural networks for the probability function, showing on two text corpora that the proposed approach significantly improves on state-of-the-art n-gram models, and that the proposed approach allows to take advantage of longer contexts.

The key intuition of this work was to turn the task of LM into a task of **latent representation learning** - that is, to learn how to encode sequences of tokens into a rich representation.

Historically, this approach has led to a breakthrough in performance (as measured by perplexity) for the LM objective. The Transformer architecture has embodied a scalable neural network method that led to performance that is better than that humans on next-token prediction even with "small" models.

Language models are better than humans at next-token prediction

<https://arxiv.org/abs/2212.11281>

[Buck Shlegeris](#), [Fabien Roger](#), [Lawrence Chan](#), [Euan McLean](#)

TMLR 2024

Current language models are considered to have sub-human capabilities at natural language tasks like question-answering or writing code. However, language models are not trained to perform well at these tasks, they are trained to accurately predict the next token given previous tokens in tokenized text. It is not clear whether language models are better or worse than humans at next token prediction. To try to answer this question, we performed two distinct experiments to directly compare humans and language models on this front: one measuring top-1 accuracy and the other measuring perplexity. In both experiments, we find humans to be consistently **worse** than even relatively small language models like GPT3-Ada at next-token prediction.

Transformers

The **Transformer architecture** has improved the performance of neural language models in a very significant manner compared to previous neural architectures (RNNs, LSTMs, GRUs) which were developed as a replacement of n-gram models. In this approach, the probability p is modeled using a transformer architecture with parameters θ . For example, GPT uses a multi-layer transformer decoder for the language model. This model applies a multi-head self-attention operation over the input context tokens followed by position-wise feedforward layers to produce an output distribution over target tokens. The transformer architecture includes the following components, adopting the notation used in [Radford et al, 2018]:

- Initial token embedding: $h_0 = UW_e + W_p$

- Multi-layer transformation: $h_l = \text{transformer_block}(h_{l-1})$ for $l \in [1, n]$
- Final prediction for the next token: $p(u) = \text{softmax}(h_n W_e^T)$

where $U = (u_{-k}, \dots, u_{-1})$ is the context vector of the last k tokens, n is the number of layers, W_e is the token embedding matrix, and W_p is the position embedding matrix. The softmax function turns a vector of size V (number of distinct tokens in the tokenizer vocabulary) into a distribution (a vector of V dimensions with all values between 0 and 1 and the sum of the values is 1).

All the parameters (W_e, W_p) and those of the transformer block are trained by the end to end optimization of the LM over the corpus.

[02.5 Transformers](#) provides more technical details on this architecture.